

Credit Card Approval Prediction

Project Name	Credit Card Approval Prediction
Domain	Finance / Banking - Machine Learning
Team ID	

Project Description

Banks and financial institutions receive thousands of credit card applications every day, a significant portion of which are rejected due to factors such as high existing loan balances, insufficient income levels, or excessive credit inquiries. Manually reviewing each application is both time-consuming and prone to human error, making it an inefficient process at scale.

This project automates the credit card approval decision using machine learning. Applicant demographic and financial data (from the **application_record** dataset) is combined with historical repayment behaviour (from the **credit_record** dataset) to build a target label indicating whether an applicant is a good or bad credit risk. Four classification algorithms - Logistic Regression, Decision Tree, Random Forest, and XGBoost - are trained and compared, and the selected model is serialized and served through a Flask web application so that a prediction can be generated instantly for any new applicant.

Scenarios

Scenario 1: Automated Credit Card Application Screening

A bank credit analyst enters a new applicant's financial profile — including gender, car and property ownership, income, income type, education, family status, housing type, age, and employment duration — into the web application. The model returns an instant approval or rejection prediction, enabling the analyst to prioritize applications that require further manual review.

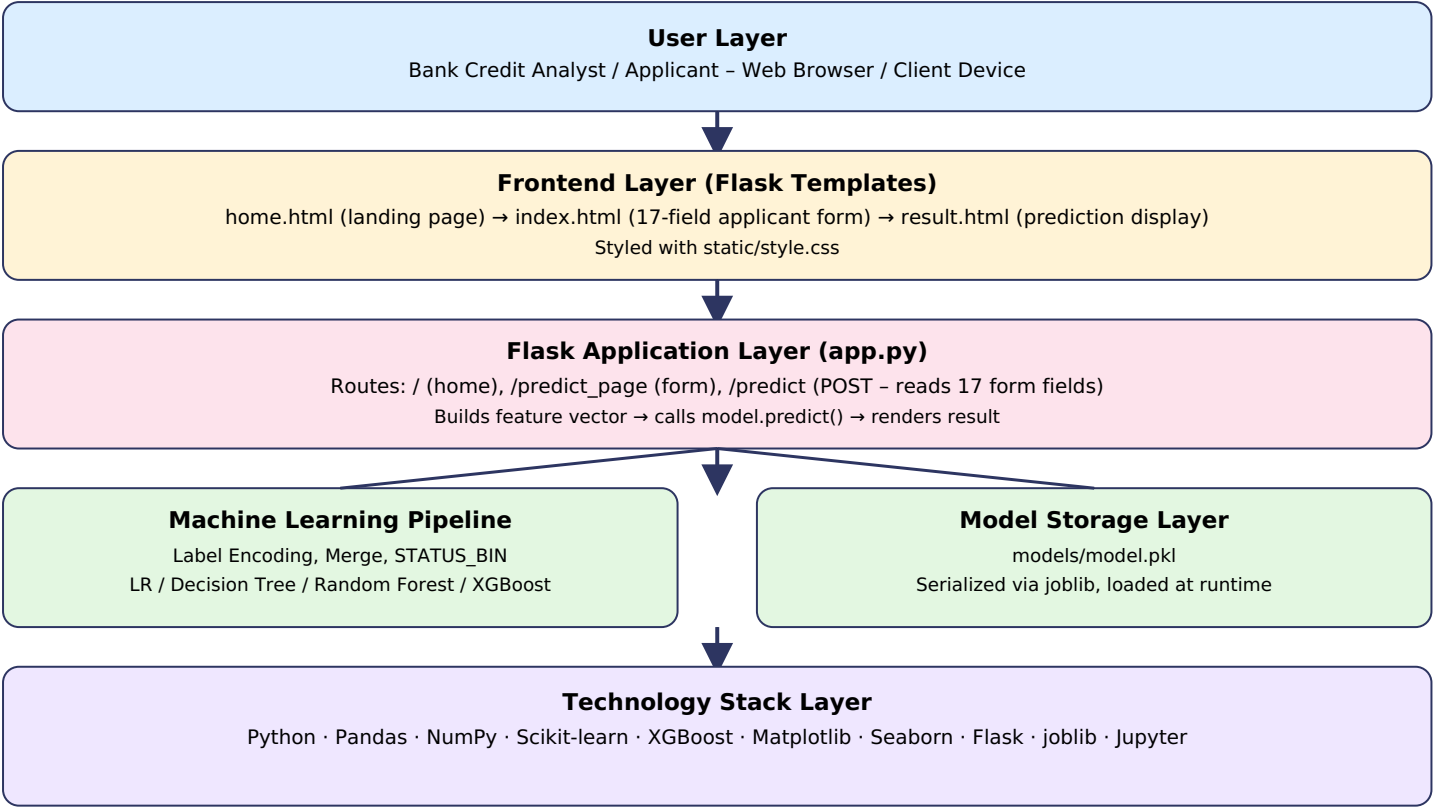
Scenario 2: High-Risk Applicant Identification

The historical credit record dataset contains monthly repayment status codes (on-time, overdue by various durations, written off, etc.). These multi-class codes are converted into a binary risk label (good-standing vs. past-due/defaulted), allowing the model to clearly flag applicants who are ineligible for a new credit card based on their repayment history.

Scenario 3: Customer Self-Service Eligibility Check

A prospective customer uses the web application to enter their own financial and demographic details before submitting a formal application. The system instantly predicts the likelihood of approval, helping the customer understand their eligibility and reducing unnecessary application rejections.

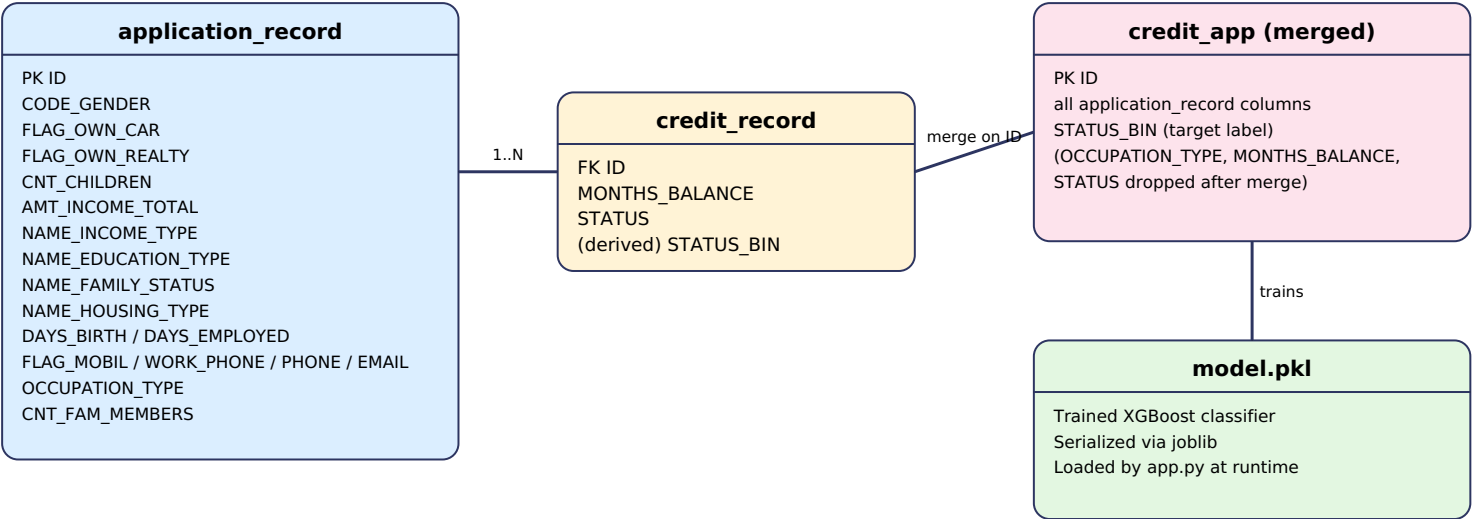
Technical Architecture



Description

The architecture diagram represents the complete workflow of the Credit Card Approval Prediction system. It consists of the User Layer, Frontend Layer, Flask Application Layer, Machine Learning Pipeline, Model Storage Layer, and Technology Stack Layer. Users provide applicant details through a web form, which the Flask application converts into a feature vector. The machine learning pipeline performs data cleaning, label encoding, and model training/comparison across four algorithms. The best-performing model is serialized with joblib and loaded at runtime for instant predictions, which are then rendered back to the user through the web interface.

Entity Relationship Diagram



Description

The Entity Relationship Diagram illustrates how the two raw datasets are combined. Each applicant in **application_record** (identified by **ID**) can have many monthly entries in **credit_record**. The two tables are merged on **ID** to form **credit_app**, and the multi-class **STATUS** code is converted into a binary **STATUS_BIN** target label (1 = good standing, 0 = overdue/past-due). After dropping identifier and leakage-prone columns, the resulting feature set is used to train the classification models, and the best-performing model is persisted as **model.pkl** for use by the Flask application.

Pre-requisites

- Python

Jupyter Notebook

Visual Studio Code

NumPy

Pandas

Scikit-learn

XGBoost

Matplotlib

Seaborn

Flask

joblib

Git & GitHub
- Kaggle (Credit Card Approval Prediction Dataset)

Project Workflow

The project is developed through a structured workflow consisting of multiple epics, covering data collection, cleaning, exploratory analysis, model development, and deployment. Each epic below reflects the actual notebook (**credit_card_prediction.ipynb**) and application code (**app.py**) in this project.

Epic 1: Environment Setup and Package Installation

Story 1: Install Required Packages

pandas, **numpy** – data loading and manipulation. **matplotlib**, **seaborn** – visualization. **scikit-learn** – preprocessing, train/test split, and three of the four classifiers. **xgboost** – gradient boosting classifier. **joblib** – model serialization. **flask** – web application framework.

```
pip install pandas numpy matplotlib seaborn scikit-learn xgboost joblib flask
```

Epic 2: Importing Required Libraries

Story 1: Required Libraries Import

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

import xgboost as xgb
import joblib
import warnings

warnings.filterwarnings("ignore")
```

Epic 3: Dataset Download and Understanding

Story 1: Reading the Dataset

The dataset consists of two Kaggle CSV files: **application_record.csv** (applicant demographic and financial details) and **credit_record.csv** (monthly repayment status history).

```
app = pd.read_csv("../dataset/application_record.csv")
credit = pd.read_csv("../dataset/credit_record.csv")
```

Story 2: Understanding the Dataset

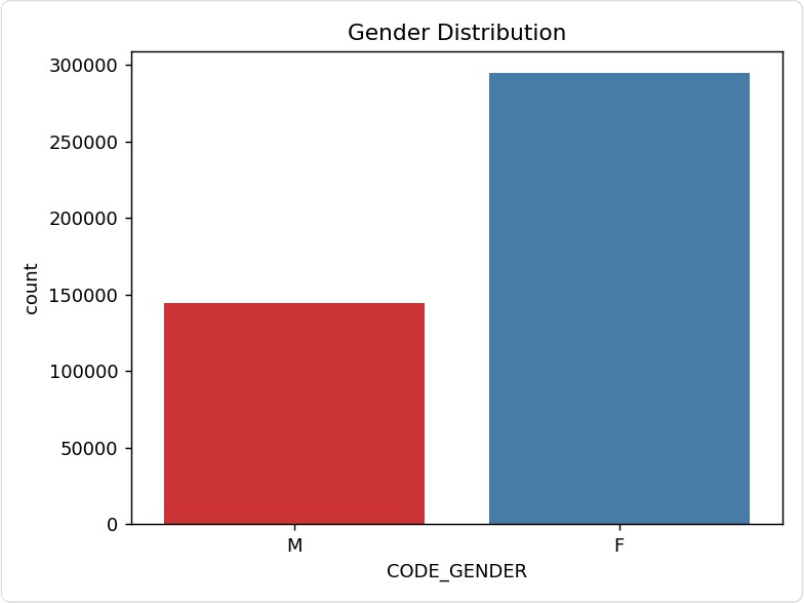
Dataset	Shape
application_record.csv	438,557 rows × 18 columns
credit_record.csv	1,048,575 rows × 3 columns

Both datasets were checked for null values and duplicate rows using **isnull().sum()** and **duplicated().sum()** before proceeding to analysis.

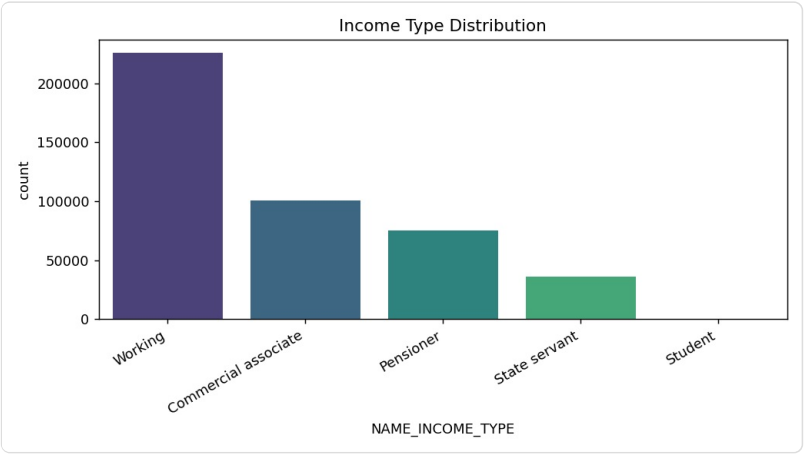
Story 3: Data Visualization

Exploratory analysis was performed on the raw applicant data to understand distributions and relationships between features.

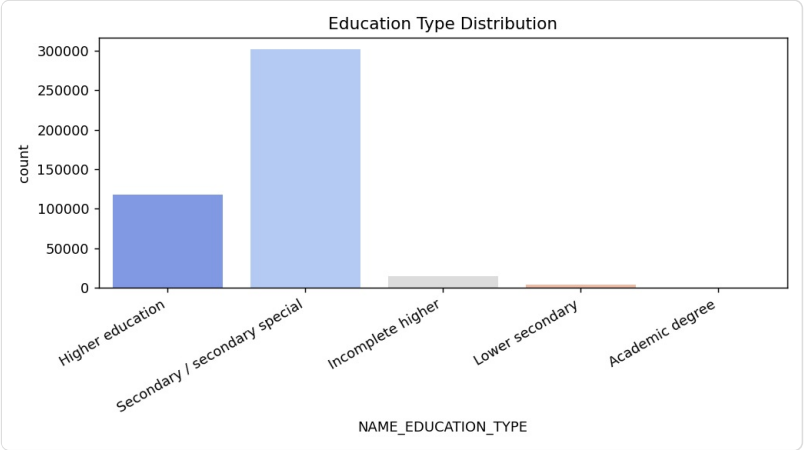
Gender Distribution



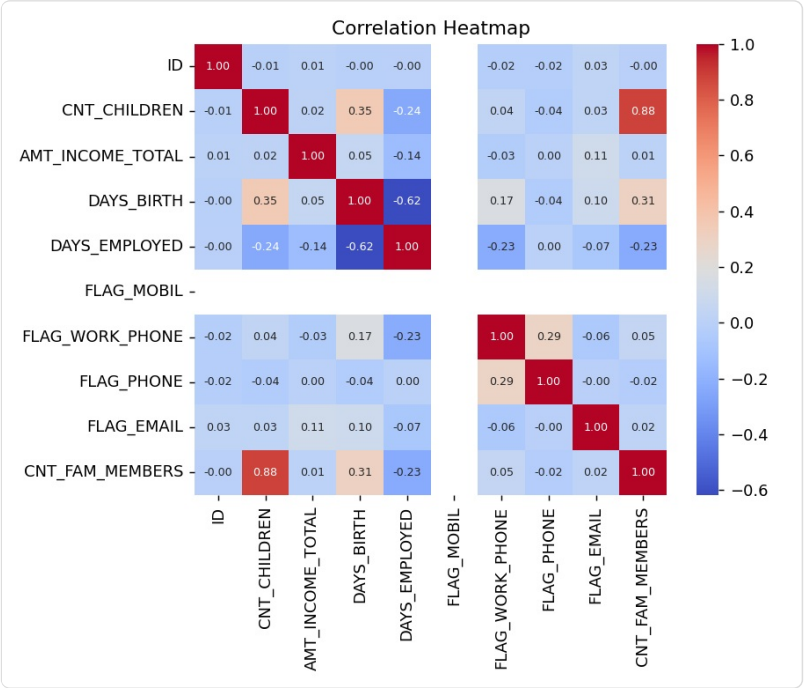
Income Type Distribution



Education Type Distribution



Correlation Heatmap



Higher correlation with the target indicates a stronger predictor and helps guide feature selection.

Epic 4: Data Cleaning and Feature Engineering

Story 1: Removing Duplicates and Fixing Sign Conventions

```
app.drop_duplicates(inplace=True)

# Convert negative values into positive values
app['DAYS_BIRTH'] = app['DAYS_BIRTH'].abs()
app['DAYS_EMPLOYED'] = app['DAYS_EMPLOYED'].abs()
```

Story 2: Converting Repayment STATUS into a Binary Target

The raw **STATUS** column in credit_record.csv is multi-class (0, 1, 2, 3, 4, 5, C, X). It is converted into a binary risk label: **1** for good standing (statuses 0, X, C) and **0** for overdue/past-due.

```
def to_binary(status):
    if status in ['0', 'X', 'C']:
        return 1      # Approved / Good standing
    else:
        return 0      # Not Approved / Past-due

credit['STATUS_BIN'] = credit['STATUS'].apply(to_binary)
```

STATUS_BIN	Count
1 (Good standing)	1,034,381
0 (Overdue / Past-due)	14,194

Story 3: Merging the Two Datasets

```
credit_app = pd.merge(app, credit, on='ID', how='left')
# Merged Dataset Shape: (1,179,815, 21)

credit_app.drop(columns=['OCCUPATION_TYPE', 'MONTHS_BALANCE', 'STATUS'], inplace=True)
credit_app.dropna(subset=['STATUS_BIN'], inplace=True)
```

Epic 5: Label Encoding

Story 1: Encoding Categorical Columns

Categorical columns are converted to numeric form using **LabelEncoder** so they can be consumed by the machine learning models.

```
gender = LabelEncoder()
car = LabelEncoder()
reality = LabelEncoder()
income = LabelEncoder()
education = LabelEncoder()
family = LabelEncoder()
housing = LabelEncoder()

credit_app["CODE_GENDER"] = gender.fit_transform(credit_app["CODE_GENDER"])
credit_app["FLAG_OWN_CAR"] = car.fit_transform(credit_app["FLAG_OWN_CAR"])
credit_app["FLAG_OWN_REALTY"] = reality.fit_transform(credit_app["FLAG_OWN_REALTY"])
credit_app["NAME_INCOME_TYPE"] = income.fit_transform(credit_app["NAME_INCOME_TYPE"])
credit_app["NAME_EDUCATION_TYPE"] = education.fit_transform(credit_app["NAME_EDUCATION_TYPE"])
credit_app["NAME_FAMILY_STATUS"] = family.fit_transform(credit_app["NAME_FAMILY_STATUS"])
credit_app["NAME_HOUSING_TYPE"] = housing.fit_transform(credit_app["NAME_HOUSING_TYPE"])
```

Project: Credit Card Approval Prediction

Epic 6: Train-Test Split

Story 1: Splitting Features and Target

```
X = credit_app.drop(["STATUS_BIN"], axis=1)
y = credit_app["STATUS_BIN"]
# Features Shape: (777715, 17)    Target Shape: (777715,)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)
# Training Features : (622172, 17)    Testing Features : (155543, 17)
```

Epic 7: Model Building and Evaluation

Story 1: Training and Comparing Four Classifiers

```
lr = LogisticRegression(max_iter=1000, random_state=42)
lr.fit(X_train, y_train)

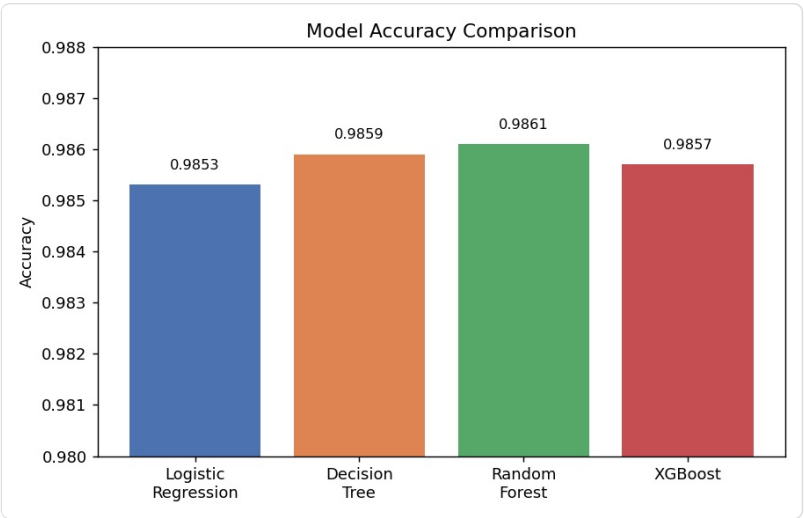
dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)

rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)

xgb_model = xgb.XGBClassifier(random_state=42, eval_metric="logloss")
xgb_model.fit(X_train, y_train)
```

Story 2: Accuracy Comparison

Model	Accuracy
Logistic Regression	0.9853
Decision Tree	0.9859
Random Forest	0.9861
XGBoost	0.9857



The dataset target is heavily imbalanced (1,034,381 good-standing vs. 14,194 past-due records), so all four models achieve high raw accuracy; precision/recall on the minority class (as seen in the classification reports) is the more meaningful indicator of real-world performance and a candidate for future improvement (e.g. class-balancing techniques).

Epic 8: Saving the Model

Story 1: Serializing the Trained Model

The trained model is serialized with **joblib** so it can be reused for predictions without retraining. The saved file is loaded directly inside the Flask application at runtime.

```
joblib.dump(xgb_model, "../models/model.pkl")
print("Model saved successfully!")

loaded_model = joblib.load("../models/model.pkl")
print(loaded_model)
```

Epic 9: Building the Flask Web Application

Story 1: Writing the Flask Backend (app.py)

```
from flask import Flask, render_template, request
import numpy as np
import joblib

app = Flask(__name__)

# Load trained model
model = joblib.load("models/model.pkl")

@app.route("/")
def home():
    return render_template("home.html")

@app.route("/predict_page")
def predict_page():
    return render_template("index.html")

@app.route("/predict", methods=["POST"])
def predict():

    features = [
        float(request.form["CODE_GENDER"]),
        float(request.form["FLAG_OWN_CAR"]),
        float(request.form["FLAG_OWN_REALTY"]),
        float(request.form["CNT_CHILDREN"]),
        float(request.form["AMT_INCOME_TOTAL"]),
        float(request.form["NAME_INCOME_TYPE"]),
        float(request.form["NAME_EDUCATION_TYPE"]),
        float(request.form["NAME_FAMILY_STATUS"]),
        float(request.form["NAME_HOUSING_TYPE"]),
        float(request.form["DAYS_BIRTH"]),
        float(request.form["DAYS_EMPLOYED"]),
        float(request.form["FLAG_MOBIL"]),
        float(request.form["FLAG_WORK_PHONE"]),
        float(request.form["FLAG_PHONE"]),
        float(request.form["FLAG_EMAIL"]),
        float(request.form["CNT_FAM_MEMBERS"]),
        float(request.form["ID"])
    ]

    prediction = model.predict([features])

    if prediction[0] == 1:
        result = "✓ Credit Card Approved"
    else:
        result = "✗ Credit Card Rejected"

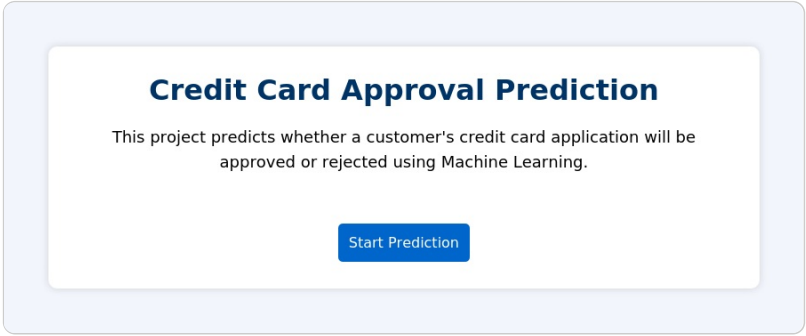
    return render_template("result.html", prediction=result)

if __name__ == "__main__":
    app.run(debug=True)
```

Project: Credit Card Approval Prediction

Story 2: Running the Application - Web Pages

Home Page (home.html) - the entry point of the application. It introduces the project and links to the prediction form.



Prediction Page (index.html) - a 17-field form covering gender, car/property ownership, income, income type, education, family status, housing type, age, employment duration, and contact flags. On submit, the values are POSTed to **/predict**.

Credit Card Approval Prediction

ID

Gender (0 = Female, 1 = Male)

Own Car (0 = No, 1 = Yes)

Own House (0 = No, 1 = Yes)

Children

Annual Income

Income Type

Education Type

Family Status

Housing Type

Age (Days Birth)

Employment Days

Mobile (Always 1)

1

Work Phone

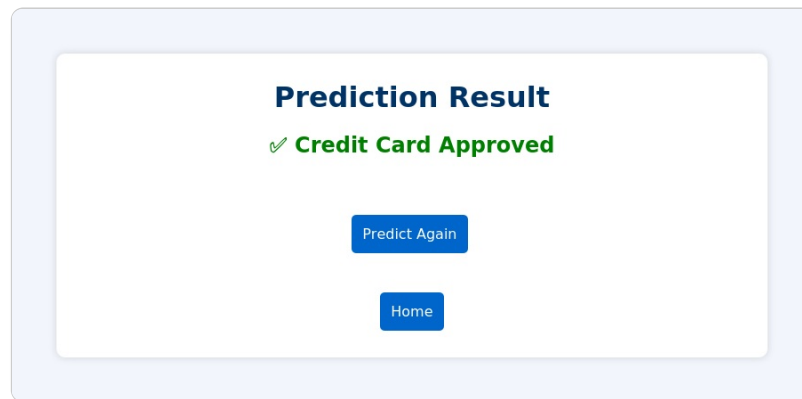
Phone

Email

Family Members

Predict

Result Page (result.html) - displays the model's prediction (Approved / Rejected) and lets the user return to the form or the home page.



Project: Credit Card Approval Prediction

Conclusion

The Credit Card Approval Prediction system successfully demonstrates the application of machine learning to automate a traditionally manual banking decision. Applicant demographic and financial data from **application_record.csv** is combined with historical repayment behaviour from **credit_record.csv** to build a binary target label, and four classification algorithms – Logistic Regression, Decision Tree, Random Forest, and XGBoost – were trained and compared, achieving accuracies between 98.5% and 98.6% on the held-out test set.

The trained model is serialized with joblib and served through a Flask web application consisting of a home page, a 17-field applicant form, and a result page, enabling both bank staff and prospective applicants to receive an instant approval/rejection prediction. The project follows a complete machine learning lifecycle – environment setup, data cleaning, exploratory analysis, label encoding, model training and evaluation, serialization, and deployment through a web interface.

Because the target classes are highly imbalanced, future improvements could include class-balancing techniques (e.g. SMOTE or class-weighting), hyperparameter tuning, and richer feature engineering to improve minority-class (past-due) detection, along with cloud deployment for scalable, real-time access.

Project: Credit Card Approval Prediction